Cuprins

1 Retele Neuronale **4**
1.1 De la neuron la perceptronul 4
1.2 Procesul de invatare al perceptronului 4
1.3 Limitarile perceptronului simplu: Problema XOR 5

2 Functii de Activare **6**
2.1 De ce sunt necesare functiile de activare? 6
2.2 Evolutia functiilor de activare 6
2.2.1 Functia Step (Heaviside) 6
2.2.2 Sigmoid: Prima functie diferentiabila 6
2.2.3 Tanh: Versiunea centrata 7
2.2.4 ReLU: Revolutia simplicitatii 7
2.2.5 Variante ReLU: Rezolvarea problemelor 8
2.3 Functii de Activare Avansate 8
2.3.1 Swish 8
2.3.2 Mish 9
2.3.3 GELU (Gaussian Error Linear Unit) 9

3 Functii de Pierdere si Optimizatori **10**
3.1 Functii de Pierdere: Masurarea Performantei 10
3.1.1 Mean Squared Error (MSE) - pentru regresie 10
3.1.2 Cross-Entropy Loss - pentru clasificare 10
3.1.3 Focal Loss 10
3.1.4 Dice Loss 11
3.1.5 IoU Loss 11
3.1.6 Triplet Loss 11
3.2 Optimizatori: Cautarea Minimului Global 11
3.2.1 Gradient Descent clasic 11
3.2.2 Stochastic Gradient Descent (SGD) 11
3.2.3 SGD cu Momentum 12
3.3 RMSProp 12
3.4 Adam (Adaptive Moment Estimation) 12
3.5 AdamW 13
3.6 Learning Rate Scheduling: Optimizarea convergentei 13

4 Forward si back propagation **14**
4.1 Forward Propagation: Calculul predictiei 14
4.2 Backpropagation: Algoritmul care a revolutionat AI 14
4.3 Exemplu pas cu pas: Backpropagation pentru XOR 15

5 Operatii Fundamentale in Computer Vision **17**
5.1 Convolutia: Detectarea caracteristicilor locale . . . 17
5.1.1 Padding: Controlul dimensiunii output-ului . . . 17
5.1.2 Stride: Controlul suprapunerii . . . 18
5.1.3 Dilation: Expandarea receptive field-ului . . . 18

6 Evaluarea Modelelor de Computer Vision **19**
6.1 Metrici pentru clasificare: Masurarea preciziei . . . 19
6.1.1 Metrici de baza . . . 19
6.1.2 Confusion Matrix: Vizualizarea erorilor . . . 19
6.1.3 ROC si AUC: Evaluarea la diferite threshold-uri . . . 19
6.2 Metrici pentru detectia obiectelor . . . 20
6.2.1 Intersection over Union (IoU) . . . 20
6.2.2 Mean Average Precision (mAP) . . . 21
6.3 Metrici pentru segmentare . . . 21
6.3.1 Dice Coefficient . . . 21
6.3.2 Jaccard Index (IoU pentru segmentare) . . . 21
6.4 Validarea creuata si evaluarea robusta . . . 22
6.4.1 K-Fold Cross Validation . . . 22
6.4.2 Stratified Sampling . . . 22
6.4.3 Evaluarea pe date din distributii diferite . . . 22

7 Tehnici Avansate si Optimizari **23**
7.1 Data Augmentation: Cresterea diversitatii datelor . . . 23
7.1.1 Transformari geometrice . . . 23
7.1.2 Transformari de culoare . . . 23
7.1.3 Augmentari avansate . . . 24
7.2 Transfer Learning: Reutilizarea cunostintelor . . . 24
7.2.1 Strategii de transfer learning . . . 24
7.2.2 Alegerea strategiei potrivite . . . 25
7.3 Regularizare: Prevenirea overfitting-ului . . . 25
7.3.1 Dropout: Regularizare prin eliminare aleatoare . . . 25
7.3.2 Batch Normalization: Normalizarea activarilor . . . 26
7.3.3 Weight Decay (L2 Regularization) . . . 26
7.3.4 Early Stopping . . . 26
7.4 Optimizari de performanta . . . 27
7.4.1 Mixed Precision Training . . . 27
7.4.2 Gradient Accumulation . . . 27
7.4.3 Model Parallelism si Data Parallelism . . . 27

8 Arhitecturi CNN Clasice si Moderne **28**
8.1 Evolutia CNN-urilor: De la LeNet la EfficientNet . . . 28
8.2 LeNet-5: Pionierul CNN-urilor . . . 28
8.3 AlexNet: Revolutia Deep Learning . . . 29
8.4 VGG: Simplitatea si adancimea . . . 30
8.5 ResNet: Rezolvarea problemei gradientului care dispare . . . 31

9 Layere si Componente Arhitecturale Avansate **33**
9.1 Layere Specializate YOLO . . . 33
9.1.1 C2f (Cross Stage Partial with 2 Convolutions) . . . 33
9.1.2 C3k2 (Cross Stage Partial with 3 Convolutions and k=2) . . . 33
9.1.3 C2pSA (Cross Stage Partial with Self-Attention) . . . 34
9.1.4 SPPF (Spatial Pyramid Pooling - Fast) . . . 34
9.2 Layere DenseNet si Transition . . . 35
9.2.1 Dense Block . . . 35
9.2.2 Transition Layer . . . 35
9.3 MobileNet / EfficientNet Blocks . . . 35
9.3.1 MBConv (Mobile Inverted Bottleneck Convolution) . . . 35
9.3.2 Squeeze-and-Excitation (SE) . . . 36
9.4 Darknet / CSP Blocks . . . 36
9.4.1 Darknet / CSP . . . 36
9.5 Layere Utilitare . . . 36
9.5.1 Flatten Layer . . . 36
9.5.2 Feature Pyramid Networks (FPN) . . . 37
9.6 Layere Transformer si Attention . . . 37
9.6.1 Transformer Layer . . . 37
9.6.2 Multi-Head Attention . . . 38
9.7 Layere de Pooling si Piramide . . . 38
9.7.1 SPP (Spatial Pyramid Pooling) . . . 38
9.7.2 FPN + PAN (Path Aggregation Network) . . . 39

10 Vision Transformers: Revolutia Atentiei **40**
10.1 De la NLP la Computer Vision: Adaptarea Transformers . . . 40
10.2 Patch Embedding: Transformarea imaginii in secventa . . . 40
10.3 Self-Attention: Intelegerea relatiilor globale . . . 41
10.4 Arhitectura completa ViT . . . 42
10.5 Avantajele ViT fata de CNN . . . 43
10.6 Limitarile ViT . . . 43

1 Retele Neuronale

1.1 De la neuron la perceptronul

Neuronul primeste semnale prin dendrite, le proceseaza in corpul celular si transmite rezultatul prin axon. Similar, perceptronul artificial:

- Primeste multiple intrari (ca dendritele)
- Aplica ponderi si un bias (procesarea in corpul celular)
- Foloseste o functie de activare (pragul de activare)
- Produce o iesire (ca axonul)

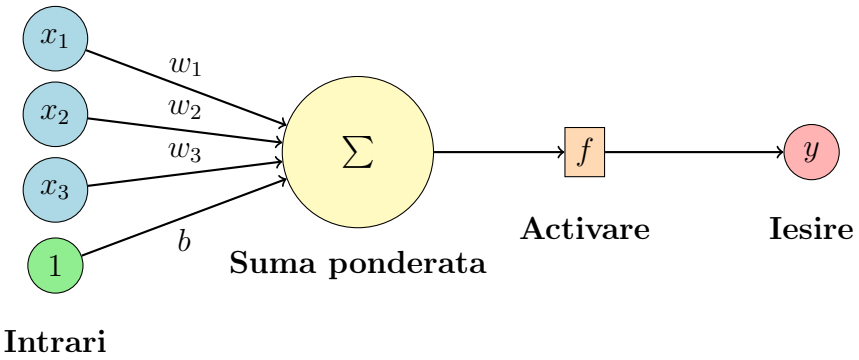


Figure 1: Structura unui perceptron cu componentele sale

Matematica perceptronului este simpla dar puternica:

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

unde:

- x_i sunt intrarile (features din datele de intrare)
- w_i sunt ponderile (parametrii invatati care determina importanta fiecarei intrari)
- b este bias-ul (permite deplasarea functiei de decizie)
- f este functia de activare (introduce non-linearitatea)

1.2 Procesul de invatare al perceptronului

Perceptronul invata prin ajustarea ponderilor bazat pe eroarea comisa. Algoritmul de invatare:

1. Initializeaza ponderile aleator: $w_i \sim \mathcal{N}(0, 0.1)$
2. Pentru fiecare exemplu de antrenare (x, y_{target}):

- Calculeaza predictia: $y_{pred} = f(\sum w_i x_i + b)$
- Calculeaza eroarea: $error = y_{target} - y_{pred}$
- Actualizeaza ponderile: $w_i = w_i + \eta \cdot error \cdot x_i$
- Actualizeaza bias-ul: $b = b + \eta \cdot error$

3. Repeta pana la convergenta

unde η este rata de invatare (learning rate), un hiperparametru crucial care controleaza cat de mari sunt pasii de actualizare.

1.3 Limitarile perceptronului simplu: Problema XOR

Un perceptron simplu poate rezolva doar probleme liniar separabile. Aceasta limitare a fost demonstrata matematic de Minsky si Papert in 1969, ducand la AI Winter.

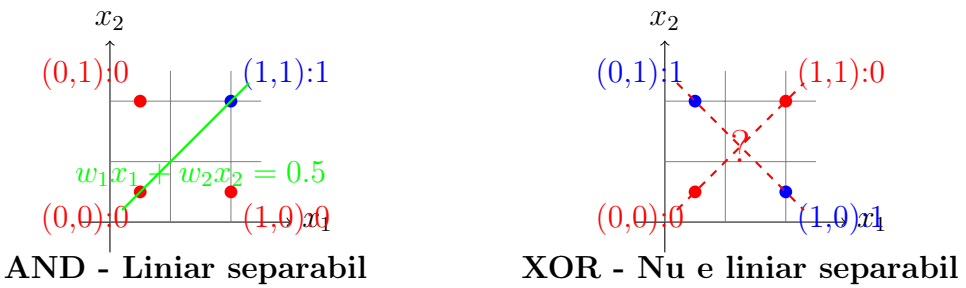
Pentru functia XOR, nu exista o linie dreapta care sa separe punctele: - $(0,0) \to 0$ si $(1,1) \to 0$ (aceeasi clasa) - $(0,1) \to 1$ si $(1,0) \to 1$ (aceeasi clasa)

Demonstratie matematica: Pentru o functie liniara $f(x_1, x_2) = w_1 x_1 + w_2 x_2 + b$, avem nevoie de:

$$w_1 \cdot 0 + w_2 \cdot 0 + b < 0 \quad \text{(pentru (0,0))} \tag{1}$$
$$w_1 \cdot 0 + w_2 \cdot 1 + b > 0 \quad \text{(pentru (0,1))} \tag{2}$$
$$w_1 \cdot 1 + w_2 \cdot 0 + b > 0 \quad \text{(pentru (1,0))} \tag{3}$$
$$w_1 \cdot 1 + w_2 \cdot 1 + b < 0 \quad \text{(pentru (1,1))} \tag{4}$$

Din (1): $b < 0$. Din (2): $w_2 + b > 0 \Rightarrow w_2 > -b > 0$. Din (3): $w_1 + b > 0 \Rightarrow w_1 > -b > 0$. Din (4): $w_1 + w_2 + b < 0 \Rightarrow w_1 + w_2 < -b$.

Dar din (2) si (3): $w_1 + w_2 > -2b > -b$, contradictie!



(a) Functia AND

(b) Functia XOR

Figure 2: Comparatie intre probleme liniar separabile si non-separabile

Solutia: Multi-Layer Perceptron (MLP) cu straturi ascunse poate rezolva XOR prin combinarea mai multor functii liniare.

2 Functii de Activare

Functiile de activare sunt esentiale pentru ca retelele neuronale sa poata invata relatii complexe, non-liniare. Fara ele, orice retea neurala, indiferent de numarul de straturi, ar fi echivalenta cu o singura transformare liniara.

2.1 De ce sunt necesare functiile de activare?

Demonstratie matematica: Fara functii de activare, o retea cu L straturi ar fi:

$$y = W_L(W_{L-1}(...W_2(W_1x + b_1) + b_2...) + b_{L-1}) + b_L \tag{5}$$
$$= W_{combined}x + b_{combined} \tag{6}$$

unde $W_{combined} = W_L W_{L-1} ... W_1$ si $b_{combined}$ este combinatia bias-urilor.

Rezultat: Orice retea liniara, indiferent de adancime, este echivalenta cu o singura transformare liniara!

2.2 Evolutia functiilor de activare

2.2.1 Functia Step (Heaviside)

Prima functie de activare folosita:

$$f(x) = \begin{cases} 1 & \text{daca } x \geq 0 \\ 0 & \text{daca } x < 0 \end{cases}$$

Avantaje:

- Simpla computational
- Interpretabila (on/off)

Dezavantaje:

- Nu este diferentiabila in $x = 0$
- Gradientul este 0 peste tot (nu permite backpropagation)
- Output-ul este binar (deci avem pierdere de informatie)

2.2.2 Sigmoid: Prima functie diferentiabila

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Derivata: $\sigma'(x) = \sigma(x)(1 - \sigma(x))$

Avantaje:

- Diferentiabila peste tot
- Iesire in intervalul (0,1) - interpretabila ca probabilitate
- Smooth transition intre 0 si 1

- Derivata simpla de calculat

Dezavantaje:

- **Vanishing gradient:** Pentru $|x| > 4$, $\sigma'(x) \approx 0$
- Nu este centrata in zero (mean $\neq 0$)
- Computational costisitoare (exponentiala)
- Saturatie pentru valori extreme

2.2.3 Tanh: Versiunea centrata

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1$$

Derivata: $\tanh'(x) = 1 - \tanh^2(x)$
Avantaje fata de sigmoid:

- Centrata in zero: $\tanh(0) = 0$
- Interval de iesire: $(-1, 1)$
- Gradiente mai mari decat sigmoid

Dezavantaje:

- Tot sufere de vanishing gradient
- Tot este costisitoare

2.2.4 ReLU: Revolutia simplicitatii

$$ReLU(x) = \max(0, x) = \begin{cases} x & \text{daca } x > 0 \\ 0 & \text{daca } x \leq 0 \end{cases}$$

Derivata: $ReLU'(x) = \begin{cases} 1 & \text{daca } x > 0 \\ 0 & \text{daca } x \leq 0 \end{cases}$
De ce a schimbat totul:

- **Este eficienta:** Doar o operatie max
- **Nu sufera de vanishing gradient** pentru $x > 0$
- **Imprastiere:** Multi neuroni sunt 0 (eficienta)
- **Convergenta mai rapida:** Gradiente constante pentru $x > 0$
- Neuronii nu au iesiri negative

Problema: "Dying ReLU" Daca un neuron primeste mereu input-uri negative, gradientul devine 0 si neuronul "moare" (nu se mai actualizeaza niciodata).
Probabilitatea ca un neuron sa moara:

$$P(neuron_mort) = P\left(\sum w_i x_i + b < 0, \forall \text{ exemple}\right)$$

2.2.5 Variante ReLU: Rezolvarea problemelor

Leaky ReLU:

$$LeakyReLU(x) = \begin{cases} x & \text{daca } x > 0 \\ \alpha x & \text{daca } x \leq 0 \end{cases}$$

unde $\alpha \in (0, 1)$, de obicei $\alpha = 0.01$.

ELU (Exponential Linear Unit):

$$ELU(x) = \begin{cases} x & \text{daca } x > 0 \\ \alpha(e^x - 1) & \text{daca } x \leq 0 \end{cases}$$

Swish (dezvoltat de Google):

$$Swish(x) = x \cdot \sigma(\beta x) = \frac{x}{1 + e^{-\beta x}}$$

GELU (Gaussian Error Linear Unit):

$$GELU(x) = x \cdot \Phi(x) = x \cdot \frac{1}{2}\left[1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right)\right]$$

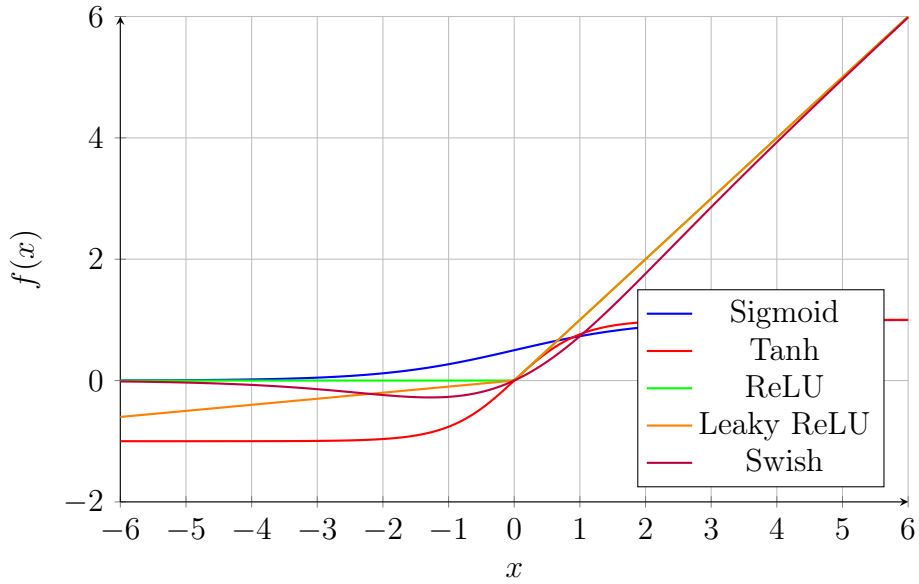


Figure 3: Comparatia functiilor de activare moderne

2.3 Functii de Activare Avansate

Pe langa functiile clasice, exista functii de activare moderne care ofera performante superioare:

2.3.1 Swish

Functia Swish, dezvoltata de Google, combina proprietatile ReLU cu o forma mai neteda:

$$\text{Swish}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

Avantaje:

- Neteda si diferentiabila peste tot
- Auto-gating: se comporta ca o functie de activare adaptiva
- Performante superioare in retele profunde

2.3.2 Mish

Functia Mish este o functie de activare auto-regularizata:

$$\text{Mish}(x) = x \cdot \tanh(\text{softplus}(x)) = x \cdot \tanh(\ln(1 + e^x))$$

Caracteristici:

- Neteda si non-monotona
- Permite gradient negativ mic pentru valori negative
- Stabilitate numerica superioara

2.3.3 GELU (Gaussian Error Linear Unit)

GELU aproximeaza functia de activare ideala pentru retele neuronale:

$$\text{GELU}(x) = x \cdot \Phi(x) = x \cdot \frac{1}{2}[1 + \text{erf}(\frac{x}{\sqrt{2}})]$$

Aproximarea practica:

$$\text{GELU}(x) \approx 0.5x(1 + \tanh[\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)])$$

Utilizata extensiv in Transformers si BERT.

3 Functii de Pierdere si Optimizatori

3.1 Functii de Pierdere: Masurarea Performantei

Functia de pierdere (loss function) masoara cat de departe sunt predictiile modelului de valorile reale. Alegerea functiei de pierdere depinde de tipul problemei.

3.1.1 Mean Squared Error (MSE) - pentru regresie

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Proprietati:

- Penalizeaza mai mult erorile mari (din cauza patratului)
- Diferentiabila peste tot
- Sensibila la outliers

Gradientul: $\frac{\partial MSE}{\partial \hat{y}_i} = \frac{2}{n}(\hat{y}_i - y_i)$

3.1.2 Cross-Entropy Loss - pentru clasificare

Binary Cross-Entropy (pentru clasificare binara):

$$BCE = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

Categorical Cross-Entropy (pentru clasificare multi-clasa):

$$CCE = -\frac{1}{n}\sum_{i=1}^{n}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c})$$

unde $y_{i,c}$ este 1 daca exemplul i apartine clasei c, altfel 0.
De ce Cross-Entropy pentru clasificare?

- Derivata din principiul maximum likelihood
- Penalizeaza mai mult predictiile foarte gresite
- Functioneaza bine cu softmax
- Gradiente mai stabile decat MSE pentru clasificare

3.1.3 Focal Loss

Pentru dezechilibrul claselor in detectia obiectelor:

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$

unde γ este parametrul de focalizare si α_t balanseaza clasele.

3.1.4 Dice Loss

Pentru segmentarea semantica:

$$\text{Dice Loss} = 1 - \frac{2|X \cap Y|}{|X| + |Y|}$$

3.1.5 IoU Loss

Pentru detectia obiectelor:

$$\text{IoU Loss} = 1 - \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

3.1.6 Triplet Loss

Pentru invatarea reprezentarilor:

$$L = \max(d(a, p) - d(a, n) + \text{margin}, 0)$$

unde a este anchor, p pozitiv, n negativ.

3.2 Optimizatori: Cautarea Minimului Global

Optimizatorii determina cum se actualizeaza parametrii modelului bazat pe gradiente.

3.2.1 Gradient Descent clasic

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

Probleme:

- Lent pentru dataset-uri mari
- Poate ramane blocat in minime locale
- Rata de invatare constanta poate fi suboptimala

3.2.2 Stochastic Gradient Descent (SGD)

Actualizeaza parametrii dupa fiecare exemplu (sau mini-batch):

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t, x_i, y_i)$$

Avantaje:

- Mult mai rapid
- Zgomotul poate ajuta sa scape din minime locale
- Poate fi paralelizat

3.2.3 SGD cu Momentum

Adauga "inertie" la actualizari:

$$v_t = \beta v_{t-1} + (1-\beta)\nabla_\theta L(\theta_t) \tag{7}$$
$$\theta_{t+1} = \theta_t - \eta v_t \tag{8}$$

Momentum-ul ajuta sa:

- Accelereze convergenta in directii consistente
- Amortizeze oscilatiile
- Treaca peste minime locale mici

3.3 RMSProp

RMSProp rezolva problema diminuarii gradientului din AdaGrad:

$$v_t = \beta v_{t-1} + (1-\beta)g_t^2$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t + \epsilon}} g_t$$

Avantaje:

- Rata de invatare adaptiva per parametru
- Nu accumuleaza gradientii la infinit
- Eficient pentru probleme non-convexe

3.4 Adam (Adaptive Moment Estimation)

Adam (Adaptive Moment Estimation) combina momentum cu adaptarea ratei de invatare:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t \quad \text{(first moment - momentum)} \tag{9}$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2 \quad \text{(second moment - adaptarea ratei)} \tag{10}$$
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \quad \text{(bias correction)} \tag{11}$$
$$\hat{v}_t = \frac{v_t}{1-\beta_2^t} \quad \text{(bias correction)} \tag{12}$$
$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{13}$$

Parametrii tipici:

- $\beta_1 = 0.9$ (momentum)
- $\beta_2 = 0.999$ (adaptarea ratei)
- $\epsilon = 10^{-8}$ (stabilitate numerica)

- $\eta = 0.001$ (learning rate)

De ce functioneaza Adam:

- Adapteaza rata de invatare pentru fiecare parametru
- Parametrii cu gradiente mari primesc rate mici
- Parametrii cu gradiente mici primesc rate mari
- Bias correction pentru primii pasi

3.5 AdamW

AdamW separa weight decay de gradient-based update:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$
$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$
$$\theta_{t+1} = \theta_t - \eta(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t)$$

Avantaje:

- Regularizare mai buna decat Adam clasic
- Convergenta mai stabila
- Performante superioare in Transformers

Click aici pentru animatie Optimizers

Figure 4: Animatie: Vizualizare Optimizers in actiune. Mai multe: AICI

3.6 Learning Rate Scheduling: Optimizarea convergentei

Rata de invatare constanta nu este mereu optimala. Scheduling-ul permite ajustarea dinamica:

Step Decay:

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/s \rfloor}$$

Exponential Decay:

$$\eta_t = \eta_0 \cdot e^{-\lambda t}$$

Cosine Annealing:

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\frac{t}{T}\pi))$$

Warm-up + Decay: Folosit in Transformers - creste rata initial, apoi o scade.

4 Forward si back propagation

4.1 Forward Propagation: Calculul predictiei

Forward propagation este procesul prin care datele trec prin retea, de la stratul de intrare la cel de iesire. Pentru o retea cu L straturi:

Pentru fiecare strat $l = 1, 2, ..., L$:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)} \quad \text{(combinatia liniara)} \tag{14}$$

$$a^{(l)} = f^{(l)}(z^{(l)}) \quad \text{(aplicarea functiei de activare)} \tag{15}$$

unde:

- $a^{(0)} = x$ (intrarea)
- $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ este matricea de ponderi pentru stratul l
- $b^{(l)} \in \mathbb{R}^{n_l}$ este vectorul de bias pentru stratul l
- $f^{(l)}$ este functia de activare pentru stratul l
- n_l este numarul de neuroni in stratul l

Exemplu concret pentru o retea 3-2-1:

$$z^{(1)} = W^{(1)}x + b^{(1)} \quad (2 \times 3 \text{ matrix}) \tag{16}$$

$$a^{(1)} = ReLU(z^{(1)}) \tag{17}$$

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} \quad (1 \times 2 \text{ matrix}) \tag{18}$$

$$a^{(2)} = \sigma(z^{(2)}) \quad \text{(output layer)} \tag{19}$$

4.2 Backpropagation: Algoritmul care a revolutionat AI

Backpropagation, dezvoltat de Rumelhart, Hinton si Williams in 1986, este algoritmul care permite retelelor neuronale sa invete prin propagarea erorii de la iesire inapoi catre intrare.

Ideea de baza: Folosim regula inlantuirii (chain rule) pentru a calcula gradientul functiei de cost in raport cu fiecare parametru.

Pentru o functie de cost L, vrem sa calculam:
$\frac{\partial L}{\partial W^{(l)}}$ - gradientul in raport cu ponderile
$\frac{\partial L}{\partial b^{(l)}}$ - gradientul in raport cu bias-urile

Pasul 1: Calculam eroarea pentru ultimul strat

$$\delta^{(L)} = \frac{\partial L}{\partial z^{(L)}} = \frac{\partial L}{\partial a^{(L)}} \odot f'(z^{(L)})$$

Pentru cross-entropy cu softmax:

$$\delta^{(L)} = a^{(L)} - y$$

Pasul 2: Propagam eroarea inapoi

$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot f'(z^{(l)})$$

Aceasta formula spune: eroarea in stratul l depinde de:

- Eroarea din stratul urmator: $\delta^{(l+1)}$
- Ponderile care conecteaza straturile: $(W^{(l+1)})^T$
- Cat de sensibil este stratul curent: $f'(z^{(l)})$

Pasul 3: Calculam gradientii

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)}(a^{(l-1)})^T$$

$$\frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}$$

Pasul 4: Actualizam parametrii

$$W^{(l)} = W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}}$$

$$b^{(l)} = b^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}}$$

4.3 Exemplu pas cu pas: Backpropagation pentru XOR

Sa consideram o retea 2-2-1 pentru a invata functia XOR:

Arhitectura: - Input: $x \in \mathbb{R}^2$ - Hidden layer: 2 neuroni cu ReLU - Output: 1 neuron cu sigmoid

Forward pass pentru $(x_1, x_2) = (1, 0)$**, target** $y = 1$**:**

Stratul ascuns:

$$z_1^{(1)} = w_{11}^{(1)} \cdot 1 + w_{12}^{(1)} \cdot 0 + b_1^{(1)} = w_{11}^{(1)} + b_1^{(1)} \tag{20}$$

$$z_2^{(1)} = w_{21}^{(1)} \cdot 1 + w_{22}^{(1)} \cdot 0 + b_2^{(1)} = w_{21}^{(1)} + b_2^{(1)} \tag{21}$$

$$a_1^{(1)} = ReLU(z_1^{(1)}), \quad a_2^{(1)} = ReLU(z_2^{(1)}) \tag{22}$$

Stratul de iesire:

$$z^{(2)} = w_1^{(2)} a_1^{(1)} + w_2^{(2)} a_2^{(1)} + b^{(2)} \tag{23}$$

$$a^{(2)} = \sigma(z^{(2)}) \quad \text{(predictia)} \tag{24}$$

Backward pass:

Eroarea la iesire:

$$\delta^{(2)} = a^{(2)} - y = a^{(2)} - 1$$

Eroarea la stratul ascuns:

$$\delta_1^{(1)} = w_1^{(2)} \delta^{(2)} \cdot ReLU'(z_1^{(1)})$$

$$\delta_2^{(1)} = w_2^{(2)} \delta^{(2)} \cdot ReLU'(z_2^{(1)})$$

Gradientii:

$$\frac{\partial L}{\partial w_1^{(2)}} = \delta^{(2)} a_1^{(1)} \tag{25}$$

$$\frac{\partial L}{\partial w_2^{(2)}} = \delta^{(2)} a_2^{(1)} \tag{26}$$

$$\frac{\partial L}{\partial w_{11}^{(1)}} = \delta_1^{(1)} \cdot 1 \tag{27}$$

$$\frac{\partial L}{\partial w_{12}^{(1)}} = \delta_1^{(1)} \cdot 0 = 0 \tag{28}$$

Click aici pentru animatie

Figure 5: Animatie: Backpropagation in actiune

5 Operatii Fundamentale in Computer Vision

5.1 Convolutia: Detectarea caracteristicilor locale

Convolutia este operatia fundamentala in procesarea imaginilor. Ea permite detectarea caracteristicilor locale precum margini, colturi, texturi prin aplicarea unor filtre (kernels) specializate.

Definitia matematica: Pentru o imagine I si un kernel K de dimensiune $(2k+1) \times (2k+1)$:

$$(I * K)(x, y) = \sum_{u=-k}^{k} \sum_{v=-k}^{v} I(x+u, y+v) \cdot K(u, v)$$

Proprietati importante:

- **Comutativitate:** $I * K = K * I$
- **Asociativitate:** $(I * K_1) * K_2 = I * (K_1 * K_2)$
- **Distributivitate:** $I * (K_1 + K_2) = I * K_1 + I * K_2$
- **Translation equivariance:** Daca translatam input-ul, output-ul se translateaza la fel

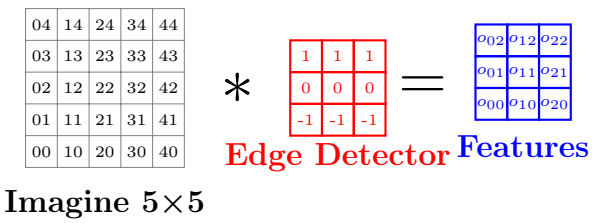


Figure 6: Operatia de convolutie 2D cu kernel de detectie margini

Tipuri comune de kernels:

(a) Sobel X (margini verticale) (b) Sobel Y (margini orizontale) (c) Box Blur (netezire) (d) Laplacian (detectie margini)

Figure 7: Kernels comuni pentru procesarea imaginilor

Parametrii convolutiei: Controlul output-ului

5.1.1 Padding: Controlul dimensiunii output-ului

Valid Padding (No Padding):

$$Output_size = Input_size - Kernel_size + 1$$

Same Padding: Adauga padding pentru a pastra dimensiunea:

$$Padding = \frac{Kernel_size - 1}{2}$$

Full Padding:

$$Padding = Kernel_size - 1$$

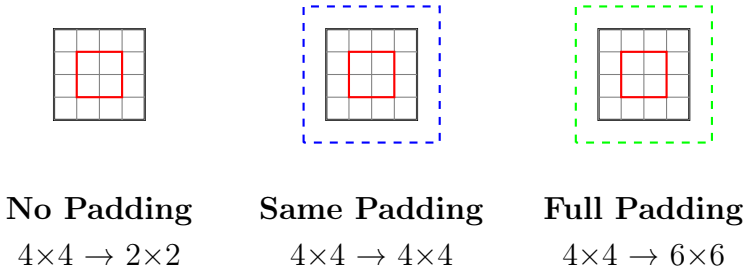


Figure 8: Tipuri de padding in convolutie

5.1.2 Stride: Controlul suprapunerii

Stride-ul determina cu cati pixeli se deplaseaza kernel-ul:

$$Output_size = \frac{Input_size + 2 \times Padding - Kernel_size}{Stride} + 1$$

Stride = 1: Suprapunere maxima, detalii fine **Stride = 2:** Reduce dimensiunea cu jumatate, mai eficient

5.1.3 Dilation: Expandarea receptive field-ului

Dilated convolution introduce "gauri" in kernel:

$$Effective_kernel_size = Kernel_size + (Kernel_size - 1) \times (Dilation - 1)$$

Avantaje:

- Receptive field mai mare fara parametrii suplimentari
- Util pentru segmentare semantica
- Pastreaza rezolutia spatiala

6 Evaluarea Modelelor de Computer Vision

6.1 Metrici pentru clasificare: Masurarea preciziei

Evaluarea corecta a modelelor este cruciala pentru intelegerea performantei reale si identificarea problemelor.

6.1.1 Metrici de baza

Accuracy (Acuratete):

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision (Precizie):

$$Precision = \frac{TP}{TP + FP}$$

Raspunde la: "Din toate predictiile pozitive, cate sunt corecte?"

Recall (Sensitivitate):

$$Recall = \frac{TP}{TP + FN}$$

Raspunde la: "Din toate cazurile pozitive reale, cate am detectat?"

F1-Score:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Media armonica intre precision si recall.

Specificitate:

$$Specificity = \frac{TN}{TN + FP}$$

Capacitatea de a identifica corect cazurile negative.

6.1.2 Confusion Matrix: Vizualizarea erorilor

		Predictii	
		Positive	**Negative**
Actual	**Positive**	**TP: 85**	**FN: 15**
	Negative	**FP: 10**	**TN: 90**

$$Accuracy = \frac{85+90}{200} = 87.5\%$$
$$Precision = \frac{85}{85+10} \approx 89.5\%$$
$$Recall = \frac{85}{85+15} = 85\%$$
$$F_1 = 2 \cdot \frac{0.895 \cdot 0.85}{0.895+0.85} \approx 87.2\%$$

Figure 9: Confusion Matrix si calculul metricilor pentru clasificare binara

6.1.3 ROC si AUC: Evaluarea la diferite threshold-uri

ROC Curve (Receiver Operating Characteristic): Graficul True Positive Rate vs False Positive Rate la diferite threshold-uri.

$$TPR = \frac{TP}{TP + FN} = Recall$$

$$FPR = \frac{FP}{FP + TN} = 1 - Specificity$$

AUC (Area Under Curve): Masoara calitatea clasificatorului independent de threshold.

- AUC = 1.0: Clasificator perfect
- AUC = 0.5: Clasificator aleator
- AUC < 0.5: Clasificator mai rau decat aleator

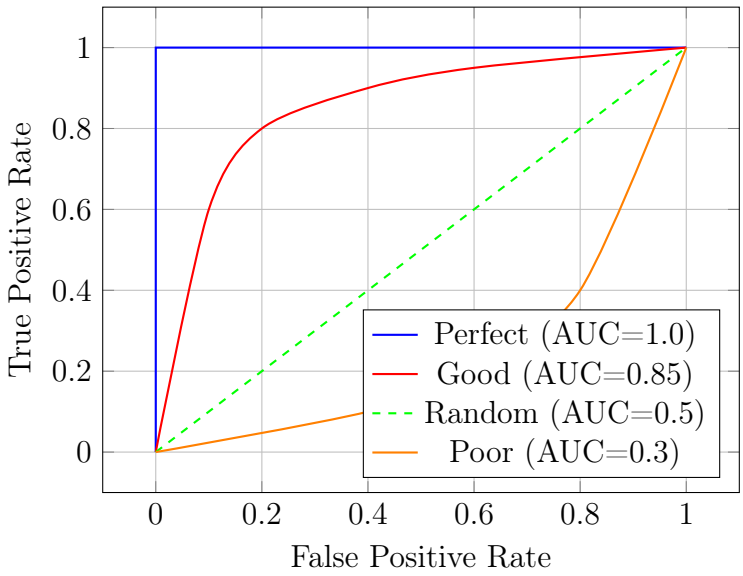


Figure 10: Comparatie valori AUC

6.2 Metrici pentru detectia obiectelor

6.2.1 Intersection over Union (IoU)

IoU masoara suprapunerea intre bounding box-ul prezis si cel real:

$$IoU = \frac{Area(B_{pred} \cap B_{gt})}{Area(B_{pred} \cup B_{gt})}$$

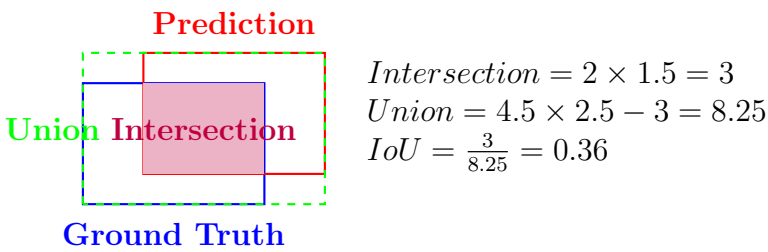


Figure 11: Calculul IoU pentru detectia obiectelor

Interpretarea IoU:

- IoU > 0.5: Detectie acceptabila
- IoU > 0.7: Detectie buna
- IoU > 0.9: Detectie excelenta

6.2.2 Mean Average Precision (mAP)

mAP este metrica standard pentru evaluarea detectiei obiectelor.

Pasii pentru calculul mAP:

1. Pentru fiecare clasa, calculeaza Average Precision (AP)
2. AP se calculeaza din curba Precision-Recall
3. mAP este media AP-urilor pentru toate clasele

$$mAP = \frac{1}{C} \sum_{c=1}^{C} AP_c$$

Variante comune:

- **mAP@0.5:** IoU threshold $= 0.5$
- **mAP@0.75:** IoU threshold $= 0.75$ (mai strict)
- **mAP@[0.5:0.95]:** Media pentru threshold-uri de la 0.5 la 0.95

6.3 Metrici pentru segmentare

6.3.1 Dice Coefficient

Folosit frecvent in segmentarea medicala:

$$Dice = \frac{2|A \cap B|}{|A| + |B|}$$

unde A este segmentarea prezisa si B este ground truth.

6.3.2 Jaccard Index (IoU pentru segmentare)

$$Jaccard = \frac{|A \cap B|}{|A \cup B|}$$

Relatie cu Dice: $Dice = \frac{2 \times Jaccard}{1 + Jaccard}$

6.4 Validarea creuata si evaluarea robusta

6.4.1 K-Fold Cross Validation

Pentru dataset-uri mici, k-fold CV ofera o evaluare mai robusta:

1. Imparte dataset-ul in k parti egale
2. Antreneaza pe k-1 parti, testeaza pe partea ramasa
3. Repeta pentru toate combinatiile
4. Media rezultatelor finale

6.4.2 Stratified Sampling

Pentru dataset-uri dezechilibrate, pastreaza proportia claselor in fiecare fold.

6.4.3 Evaluarea pe date din distributii diferite

Domain Shift: Testarea pe date din domenii diferite de antrenare.
Exemple:

- Antrenare pe imagini de zi, testare pe imagini de noapte
- Antrenare pe fotografii, testare pe desene
- Antrenare pe o camera, testare pe alta camera

7 Tehnici Avansate si Optimizari

7.1 Data Augmentation: Cresterea diversitatii datelor

Data augmentation este o tehnica esentiala pentru imbunatatirea generalizarii modelelor prin crearea de variatii artificiale ale datelor de antrenare.

7.1.1 Transformari geometrice

Rotatie:

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Aplicare: Roteste imaginea cu un unghi aleator $\theta \in [-15, 15]$

Scalare:

$$S(s_x, s_y) = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

Aplicare: Scalare uniforma $s \in [0.8, 1.2]$ sau non-uniforma

Translatie:

$$T(t_x, t_y) = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Shearing (Forfecare):

$$Sh(sh_x, sh_y) = \begin{bmatrix} 1 & sh_x \\ sh_y & 1 \end{bmatrix}$$

Flip:

- Horizontal flip: $x' = W - x$
- Vertical flip: $y' = H - y$

7.1.2 Transformari de culoare

Ajustarea luminozitatii:

$$I'(x, y) = I(x, y) + \alpha$$

unde $\alpha \in [-50, 50]$ pentru imagini cu valori [0,255]

Ajustarea contrastului:

$$I'(x, y) = \alpha \cdot I(x, y) + \beta$$

unde $\alpha \in [0.5, 1.5]$ (factor de contrast) si β este bias-ul

Modificarea saturatiei (in spatiul HSV):

$$S' = S \cdot \alpha$$

unde $\alpha \in [0.5, 1.5]$

Modificarea nuantei:

$$H' = (H + \delta) \mod 360$$

unde $\delta \in [-30, 30]$ grade

7.1.3 Augmentari avansate

Cutout: Mascare aleatoare a unor regiuni rectangulare

- Forteaza modelul sa nu se bazeze pe o singura regiune
- Simuleaza ocluzia partiala
- Dimensiunea: 10-20% din imaginea totala

Mixup: Combinarea liniara a doua imagini

$$x_{mixed} = \lambda x_i + (1 - \lambda) x_j$$

$$y_{mixed} = \lambda y_i + (1 - \lambda) y_j$$

unde $\lambda \sim Beta(\alpha, \alpha)$ cu $\alpha = 0.2$

CutMix: Combinarea regiunilor din imagini diferite

- Taie o regiune din prima imagine
- Lipeste o regiune din a doua imagine
- Eticheta este proportionala cu aria fiecarei regiuni

7.2 Transfer Learning: Reutilizarea cunostintelor

Transfer learning permite folosirea modelelor pre-antrenate pe dataset-uri mari (ImageNet) pentru task-uri specifice cu date limitate.

7.2.1 Strategii de transfer learning

1. Feature Extraction (Frozen backbone):

- Ingheata toate straturile backbone-ului
- Antreneaza doar head-ul (clasificatorul final)
- Rapid si eficient pentru dataset-uri mici
- Learning rate: 0.001-0.01

2. Fine-tuning (End-to-end):

- Antreneaza intregul model cu learning rate mic
- Permite adaptarea feature-urilor la noul domeniu
- Necesar pentru dataset-uri mai mari
- Learning rate: 0.0001-0.001 (10x mai mic decat antrenarea de la zero)

3. Progressive Unfreezing:

- Incepe cu feature extraction
- Deblocheaza treptat straturile, incepand cu ultimele

- Permite adaptarea graduala
- Reduce riscul de overfitting

4. Discriminative Learning Rates:

- Straturi diferite primesc learning rates diferite
- Straturile timpurii: LR foarte mic (features generale)
- Straturile tarziu: LR mai mare (features specifice task-ului)

7.2.2 Alegerea strategiei potrivite

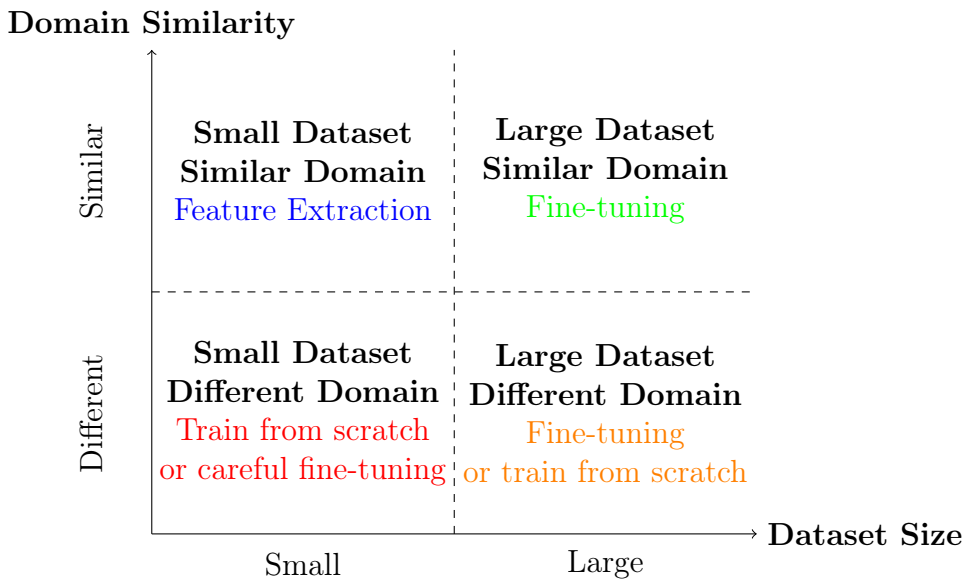


Figure 12: Alegerea strategiei de transfer learning bazat pe dimensiunea dataset-ului si similaritatea domeniului

7.3 Regularizare: Prevenirea overfitting-ului

Regularizarea previne overfitting-ul prin limitarea complexitatii modelului sau prin adaugarea de zgomot in procesul de antrenare.

7.3.1 Dropout: Regularizare prin eliminare aleatoare

$$y = \frac{1}{p} \cdot mask \odot x$$

unde $mask$ este un vector binar aleator cu probabilitatea p de a fi 1.
Cum functioneaza:

- In antrenare: Elimina aleator neuroni cu probabilitatea $(1 - p)$
- In inferenta: Foloseste toti neuronii, dar scalati cu p
- Previne co-adaptarea neuronilor

- Simuleaza un ensemble de retele

Valori tipice:

- Straturi fully connected: $p = 0.5$
- Straturi convolutionale: $p = 0.2 - 0.3$
- Input layer: $p = 0.8 - 0.9$

7.3.2 Batch Normalization: Normalizarea activarilor

$$BN(x) = \gamma \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$$

unde:

- μ_B, σ_B^2 sunt media si varianta batch-ului
- γ, β sunt parametrii invatati
- ϵ este o constanta mica pentru stabilitate numerica

Beneficii:

- Accelereaza antrenarea (permite learning rates mai mari)
- Reduce dependenta de initializarea ponderilor
- Are efect de regularizare (reduce overfitting-ul)
- Stabilizeaza distributia activarilor

7.3.3 Weight Decay (L2 Regularization)

$$L_{total} = L_{original} + \lambda \sum_i w_i^2$$

Efectul: Penalizeaza ponderile mari, fortand modelul sa foloseasca ponderi mai mici si mai distribuite.

Valori tipice: $\lambda \in [10^{-5}, 10^{-3}]$

7.3.4 Early Stopping

Opreste antrenarea cand performanta pe validation set incepe sa se degradeze.

Algoritm:

1. Monitorizeaza metrica pe validation set
2. Daca nu se imbunatateste pentru *patience* epoci
3. Opreste antrenarea si restaureaza cel mai bun model

7.4 Optimizari de performanta

7.4.1 Mixed Precision Training

Foloseste FP16 pentru forward pass si FP32 pentru backward pass:

- Reduce memoria cu 50%
- Accelereaza antrenarea cu 1.5-2x
- Mentine acuratetea numerica

7.4.2 Gradient Accumulation

Pentru batch-uri mari care nu incap in memoria GPU:

1. Imparte batch-ul mare in mini-batch-uri
2. Calculeaza gradientii pentru fiecare mini-batch
3. Acumuleaza gradientii
4. Actualizeaza parametrii dupa toate mini-batch-urile

7.4.3 Model Parallelism si Data Parallelism

Data Parallelism:

- Replica modelul pe mai multe GPU-uri
- Fiecare GPU proceseaza un subset din batch
- Sincronizeaza gradientii la sfarsitul fiecarui pas

Model Parallelism:

- Imparte modelul pe mai multe GPU-uri
- Fiecare GPU calculeaza o parte din model
- Necesar pentru modele foarte mari

8 Arhitecturi CNN Clasice si Moderne

8.1 Evolutia CNN-urilor: De la LeNet la EfficientNet

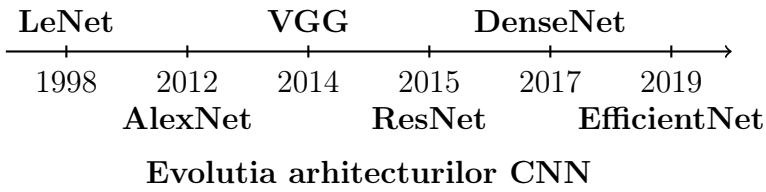


Figure 13: Timeline-ul evolutiei CNN-urilor

8.2 LeNet-5: Pionierul CNN-urilor

LeNet-5, dezvoltat de Yann LeCun in 1998, a fost primul CNN de succes pentru recunoasterea cifrelor scrise de mana (MNIST).

Arhitectura detaliata:

1. **Input:** 32×32×1 (imagini grayscale)
2. **C1:** Conv 6@5×5, stride=1 → 28×28×6
3. **S2:** AvgPool 2×2, stride=2 → 14×14×6
4. **C3:** Conv 16@5×5, stride=1 → 10×10×16
5. **S4:** AvgPool 2×2, stride=2 → 5×5×16
6. **C5:** Conv 120@5×5 → 1×1×120 (fully connected)
7. **F6:** FC 84 neuroni
8. **Output:** FC 10 neuroni (cifre 0-9)

Inovatii cheie:

- Prima folosire a convolutiei pentru recunoasterea imaginilor
- Alternarea conv-pooling pentru reducerea dimensionalitatii
- Shared weights (parametrii sunt reutilizati)
- Gradient-based learning

Limitari:

- Foarte mica (60k parametrii)
- Functioneaza doar pe imagini simple
- Average pooling (mai putin eficient decat max pooling)
- Tanh activation (probleme cu vanishing gradients)

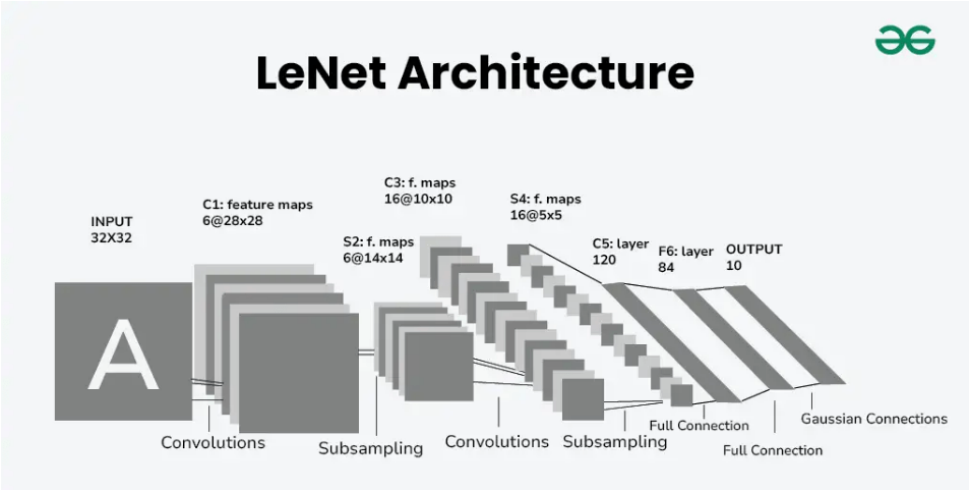


Figure 14: Arhitectura LeNet-5

8.3 AlexNet: Revolutia Deep Learning

AlexNet (2012) a castigat ImageNet cu o marja uriasa, declansand revolutia deep learning.

Inovatii:

- **ReLU activation:** Prima folosire la scara mare
- **Dropout:** Regularizare pentru prevenirea overfitting-ului
- **Data augmentation:** Crop, flip, color jittering
- **GPU training:** Paralelizare pe 2 GPU-uri
- **Local Response Normalization:** Normalizare locala

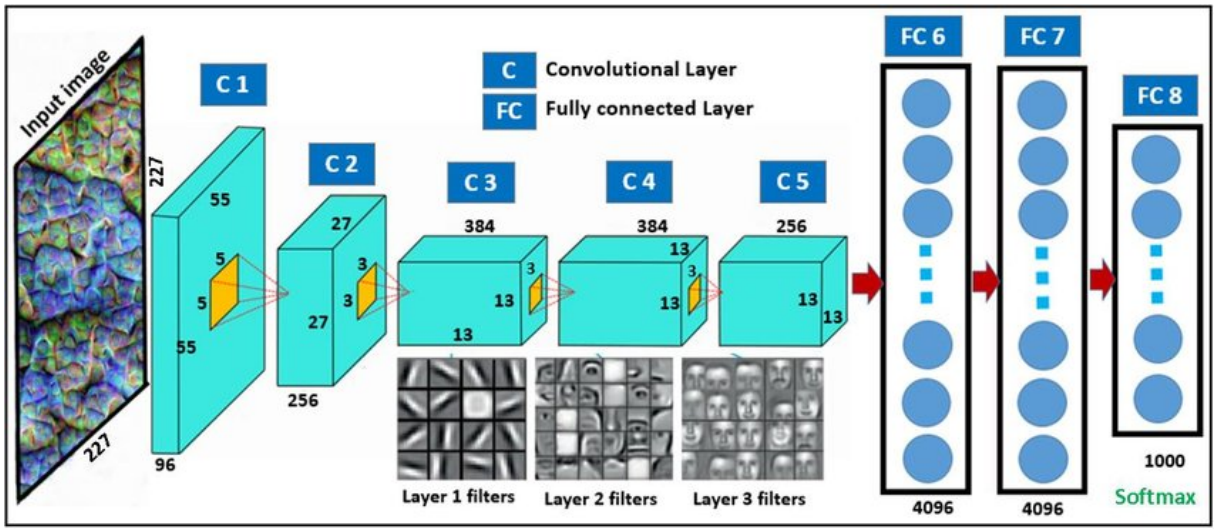


Figure 15: Arhitectura AlexNet

8.4 VGG: Simplitatea si adancimea

VGG (2014) a demonstrat ca retelele mai adanci cu filtre mici (3×3) sunt mai eficiente.

Principii de design:

- Doar convolutii 3×3 si pooling 2×2
- Dublarea numarului de filtre dupa fiecare pooling
- Arhitectura foarte regulata si simpla

VGG-16 arhitectura:

- **Block 1:** 2×Conv 64@3×3 → MaxPool
- **Block 2:** 2×Conv 128@3×3 → MaxPool
- **Block 3:** 3×Conv 256@3×3 → MaxPool
- **Block 4:** 3×Conv 512@3×3 → MaxPool
- **Block 5:** 3×Conv 512@3×3 → MaxPool
- **Classifier:** 3×FC layers

De ce 3×3 convolutions?

- Doua convolutii 3×3 = o convolutie 5×5 (acelasi receptive field)
- Mai putini parametrii: $2 \times 3^2 = 18 < 25 = 5^2$
- Mai multe non-linearitati (ReLU intre convolutii)
- Mai multa expresivitate

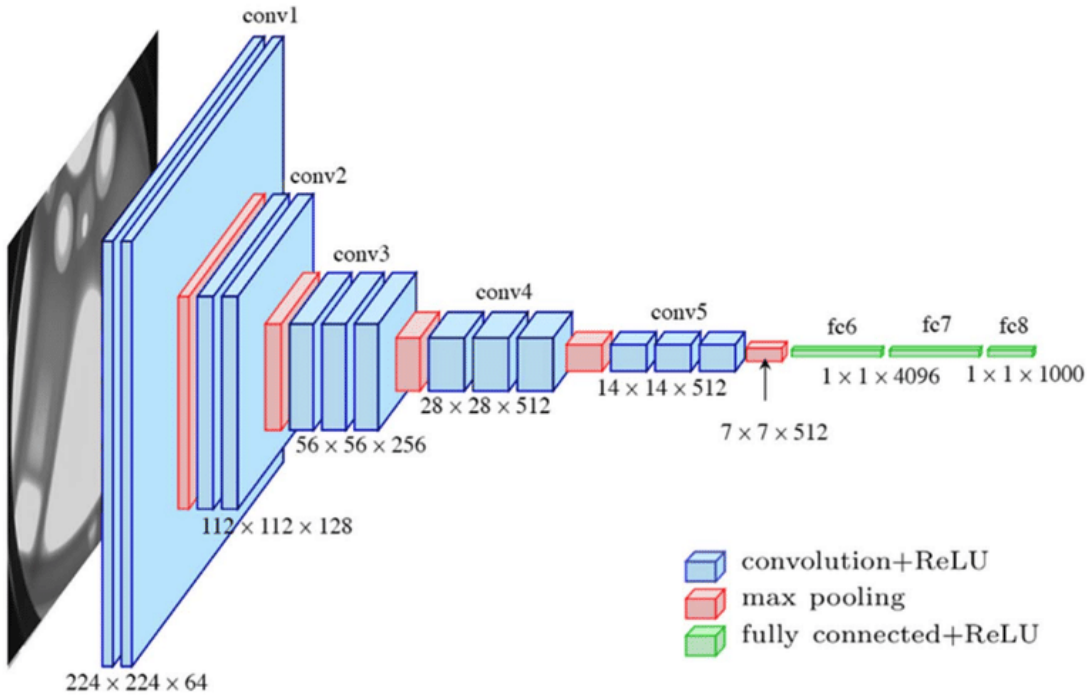


Figure 16: Arhitectura VGG

8.5 ResNet: Rezolvarea problemei gradientului care dispare

ResNet (2015) a introdus skip connections, permitand antrenarea retelelor foarte adanci (pana la 1000+ straturi).

Problema retelelor adanci: Pe masura ce adancimea creste, gradientii devin din ce in ce mai mici (vanishing gradients), facand antrenarea imposibila.

Solutia ResNet: Skip Connections

$$y = F(x, \{W_i\}) + x$$

unde $F(x, \{W_i\})$ este transformarea reziduala.

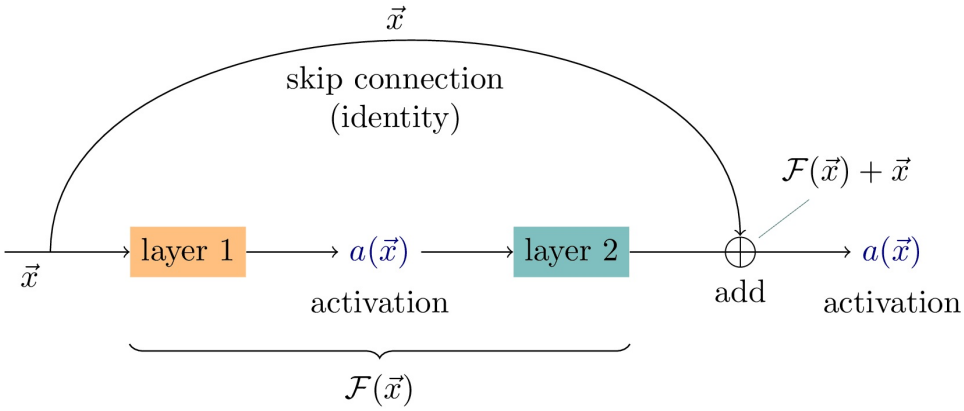


Figure 17: Blocul rezidual din ResNet cu skip connection

De ce functioneaza ResNet:

- **Gradient flow:** Skip connections permit gradientilor sa treaca direct
- **Identity mapping:** Daca $F(x) = 0$, blocul devine identitate
- **Easier optimization:** E mai usor sa inveti $F(x) = H(x) - x$ decat $H(x)$
- **Feature reuse:** Informatia din straturi anterioare e preservata

Tipuri de blocuri ResNet:
Basic Block (ResNet-18, ResNet-34):

- 2 convolutii 3×3
- BatchNorm dupa fiecare convolutie
- ReLU dupa prima BatchNorm

Bottleneck Block (ResNet-50, ResNet-101, ResNet-152):

- Conv 1×1 (reduce channels) → Conv 3×3 → Conv 1×1 (expand channels)
- Mai eficient computational
- Permite retele mai adanci

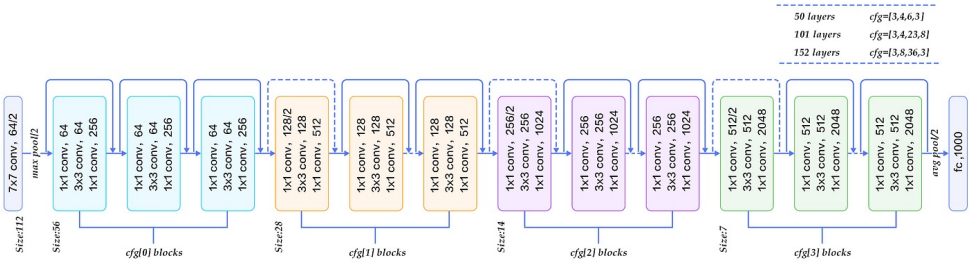


Figure 18: ResNet50: vizualizare a retelei si a skip connections

9 Layere si Componente Arhitecturale Avansate

9.1 Layere Specializate YOLO

9.1.1 C2f (Cross Stage Partial with 2 Convolutions)

C2f combina eficienta cu acuratete:

- Split input in doua parti
- Aplica convolutii pe o parte
- Concateneaza rezultatele

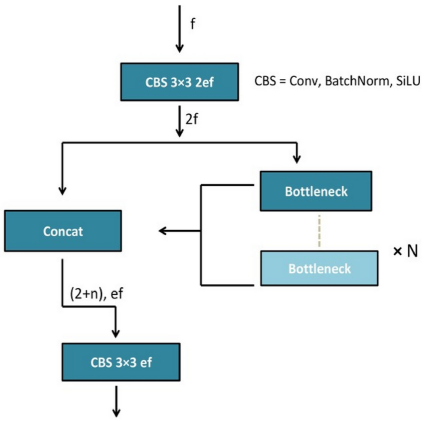


Figure 19: Layer c2f

9.1.2 C3k2 (Cross Stage Partial with 3 Convolutions and k=2)

Varianta extinsa a C2f:

- Trei convolutii in loc de doua
- Factor de repetare k=2
- Mai multe feature maps
- Acuratete superioara pentru obiecte mici

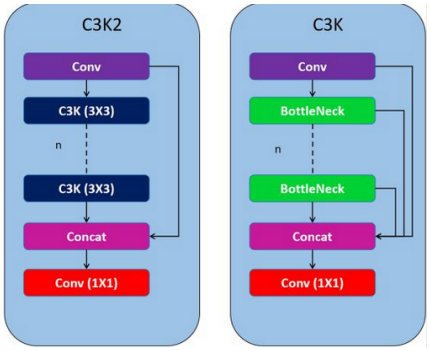


Figure 20: Layer c3k2

9.1.3 C2pSA (Cross Stage Partial with Self-Attention)

Extinde C2f cu mecanism de self-attention:

- Captureaza dependente spatiale si contextuale
- Imbunatateste detectia obiectelor complexe
- Computational overhead moderat

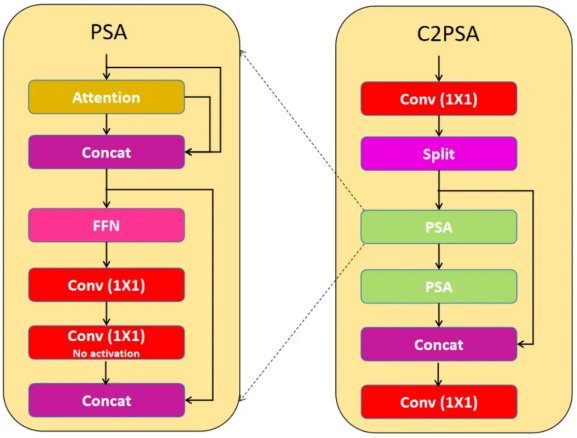


Figure 21: Layer c2psa

9.1.4 SPPF (Spatial Pyramid Pooling - Fast)

Pooling piramidal rapid:

- Permite captarea contextului multi-scale
- Imbunatateste detectia obiectelor mici si mari
- Computational eficient comparativ cu SPP clasic

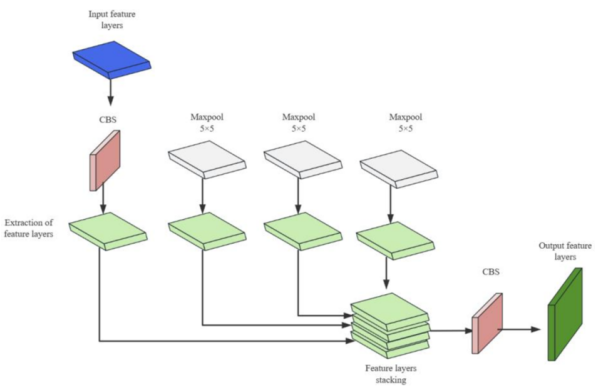


Figure 22: Layer sppf

9.2 Layere DenseNet si Transition

9.2.1 Dense Block

Fiecare layer se conecteaza la toate layerele anterioare:

$$x_l = H_l([x_0, x_1, ..., x_{l-1}])$$

- BatchNorm, ReLU, Convolutie 3x3
- Faciliteaza gradient flow si reuse feature-uri
- Reduce overfitting prin feature reuse

9.2.2 Transition Layer

Reduce dimensionalitatea intre dense blocks:

- Batch Normalization
- Convolutie 1x1 (compression)
- Average Pooling 2x2 pentru downsampling

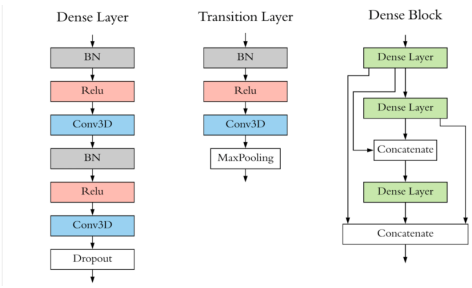


Figure 23: Transition Layer

9.3 MobileNet / EfficientNet Blocks

9.3.1 MBConv (Mobile Inverted Bottleneck Convolution)

- Expand + Depthwise Conv + Project
- Include optional Squeeze-and-Excitation (SE)
- Eficient pentru dispozitive mobile

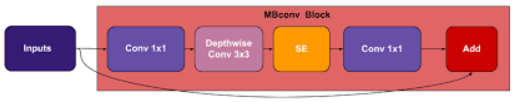


Figure 24: MBConv Block

9.3.2 Squeeze-and-Excitation (SE)

- Recalibreaza feature maps pe canale
- Modul global pooling + FC + sigmoid
- Imbunatateste acuratetea cu overhead minim

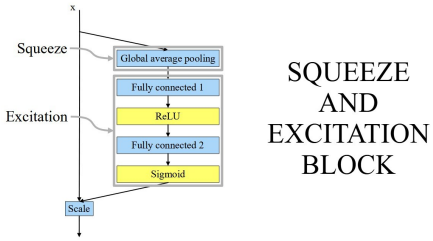


Figure 25: Squeeze-and-Excitation Layer

9.4 Darknet / CSP Blocks

9.4.1 Darknet / CSP

- Cross Stage Partial conecteaza doar o parte din input la block-ul urmator
- Reduce costul de calcul si mentine acuratetea
- Folosit in YOLOv4/v5/v8

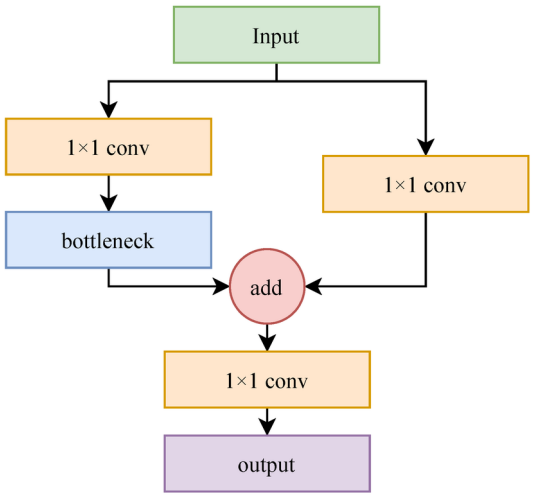


Figure 26: Darknet/CSP Block

9.5 Layere Utilitare

9.5.1 Flatten Layer

Transforma tensor multidimensional in vector:

$$\text{Input: } (N, C, H, W) \rightarrow \text{Output: } (N, C \times H \times W)$$

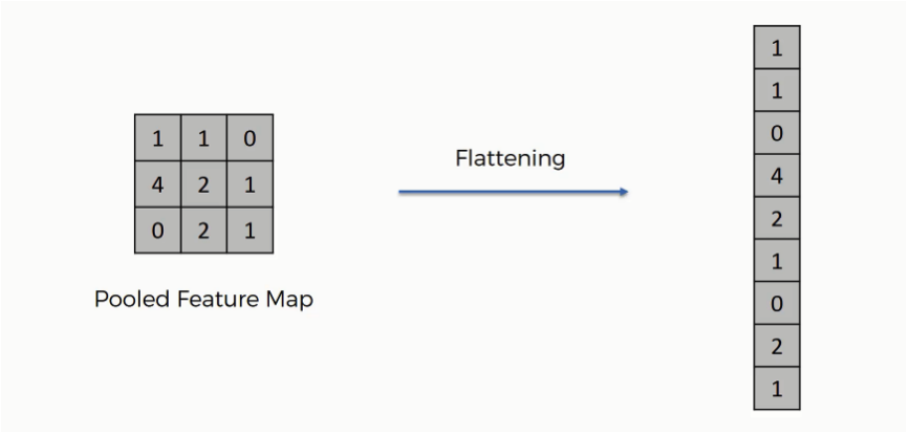


Figure 27: Flatten Layer

9.5.2 Feature Pyramid Networks (FPN)

Construieste piramida de feature-uri multi-scale:

- Top-down pathway cu lateral connections
- Combina feature-uri de la diferite scale-uri
- Esential pentru detectia obiectelor multi-scale

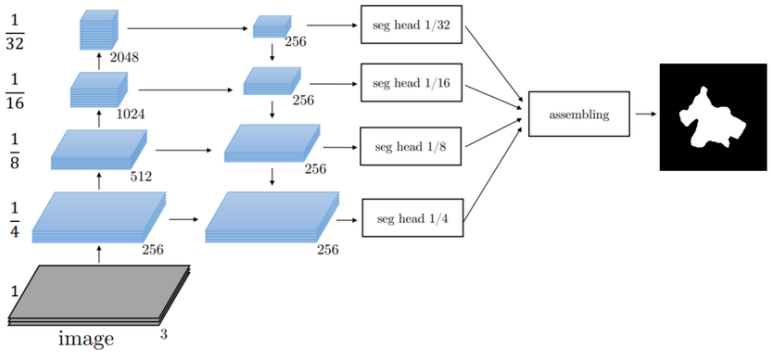


Figure 28: Feature Pyramid Network

9.6 Layere Transformer si Attention

9.6.1 Transformer Layer

Componente principale:

- Multi-Head Self Attention
- Layer Normalization
- Feed-Forward (MLP)
- Residual connections

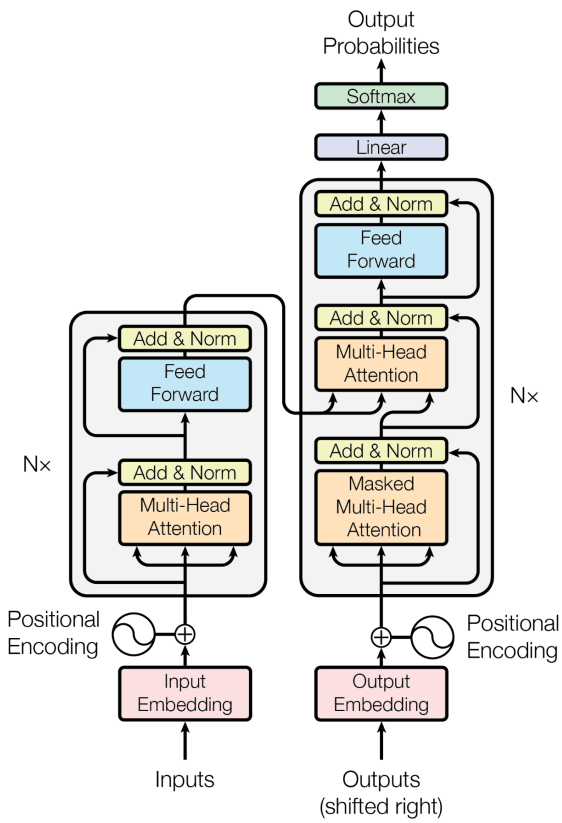


Figure 29: Transformer Layer

9.6.2 Multi-Head Attention

Extinde self-attention cu multiple capete:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_h)W^O$$

unde $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

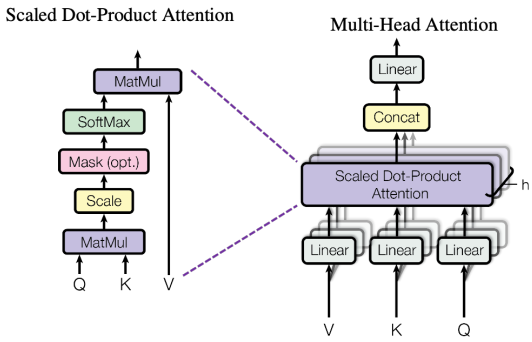


Figure 30: Multi-Head Attention

9.7 Layere de Pooling si Piramide

9.7.1 SPP (Spatial Pyramid Pooling)

- Pooling la scale multiple

- Concatenarea feature map-urilor pentru context global
- Reduce dimensiunea inputului variabil la fix

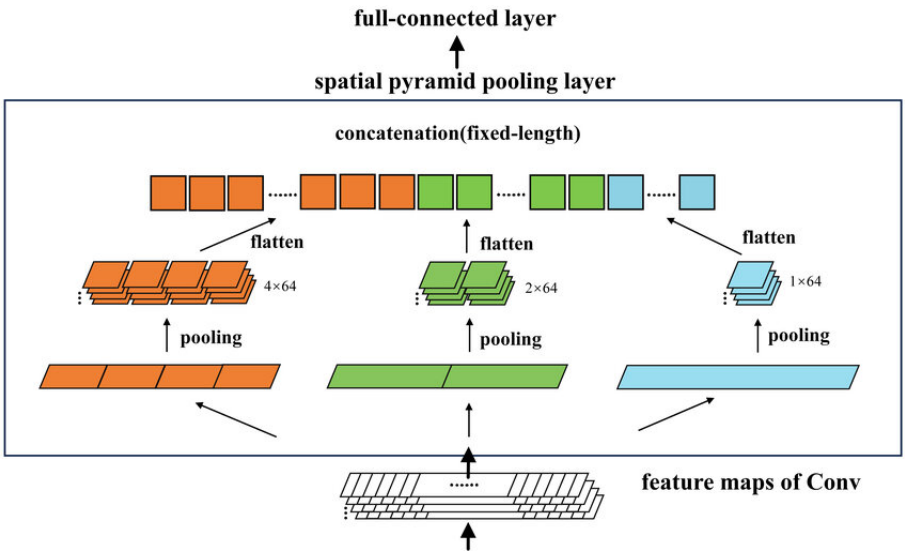


Figure 31: SPP Layer

9.7.2 FPN + PAN (Path Aggregation Network)

- FPN creeaza top-down pathway
- PAN adauga bottom-up pathway
- Imbunatateste fluxul de informatie si detectia obiectelor mici

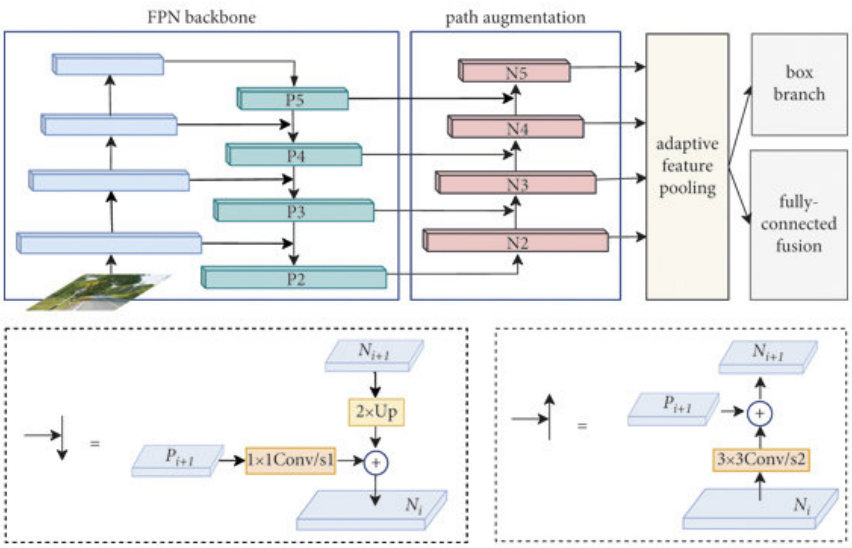


Figure 32: FPN + PAN

10 Vision Transformers: Revolutia Atentiei

10.1 De la NLP la Computer Vision: Adaptarea Transformers

Transformers au revolutionat NLP-ul cu modelele BERT si GPT. Vision Transformers (ViT) au adus aceasta revolutie si in computer vision, demonstrand ca attention-ul poate inlocui complet convolutiile.

Motivatia pentru ViT:

- CNN-urile au receptive field local (bias inductiv puternic)
- Transformers au receptive field global din primul strat
- Self-attention poate modela relatii complexe intre regiuni distante
- Scalabilitate excelenta cu datele si compute-ul

10.2 Patch Embedding: Transformarea imaginii in secventa

Prima provocare: cum transformam o imagine 2D intr-o secventa de "tokens" pentru Transformer?

Solutia: Patch Embedding

1. Impartim imaginea in patch-uri de dimensiune fixa
2. Fiecare patch devine un "cuvant" in secventa
3. Proiectam liniar fiecare patch intr-un spatiu de dimensiune fixa

Matematica patch embedding:
Pentru o imagine $x \in \mathbb{R}^{H \times W \times C}$:

1. Impartim in patch-uri: $x_p \in \mathbb{R}^{N \times (P^2 \cdot C)}$
2. unde $N = \frac{HW}{P^2}$ (numarul de patch-uri)
3. Proiectam liniar: $z_0 = [x_{class}; x_p^1 E; x_p^2 E; ...; x_p^N E] + E_{pos}$

unde:

- $E \in \mathbb{R}^{(P^2 \cdot C) \times D}$ este matricea de proiectie
- x_{class} este token-ul special pentru clasificare
- $E_{pos} \in \mathbb{R}^{(N+1) \times D}$ sunt positional embeddings

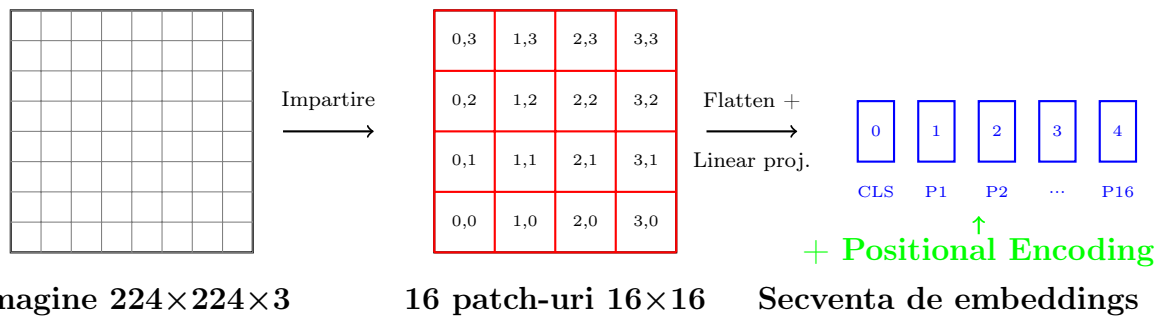


Figure 33: Transformarea imaginii in secventa de patch-uri pentru ViT, cu layout aerisit si texte clare

10.3 Self-Attention: Intelegerea relatiilor globale

Self-attention este mecanismul central al Transformers, permitand fiecarui patch sa "vada" si sa interactioneze cu toate celelalte patch-uri simultan.

Formularea matematica:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

unde:

- $Q = XW_Q$ (Query): "Ce informatii caut?"
- $K = XW_K$ (Key): "Ce informatii ofer?"
- $V = XW_V$ (Value): "Informatia pe care o transmit"
- d_k este dimensiunea key-urilor (pentru stabilitate numerica)

Multi-Head Attention:

$$MultiHead(Q, K, V) = Concat(head_1, ..., head_h)W^O$$

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

De ce multi-head?

- Fiecare head poate sa se concentreze pe aspecte diferite
- Head 1: relatii spatiale locale
- Head 2: relatii semantice globale
- Head 3: relatii de culoare/textura
- etc.

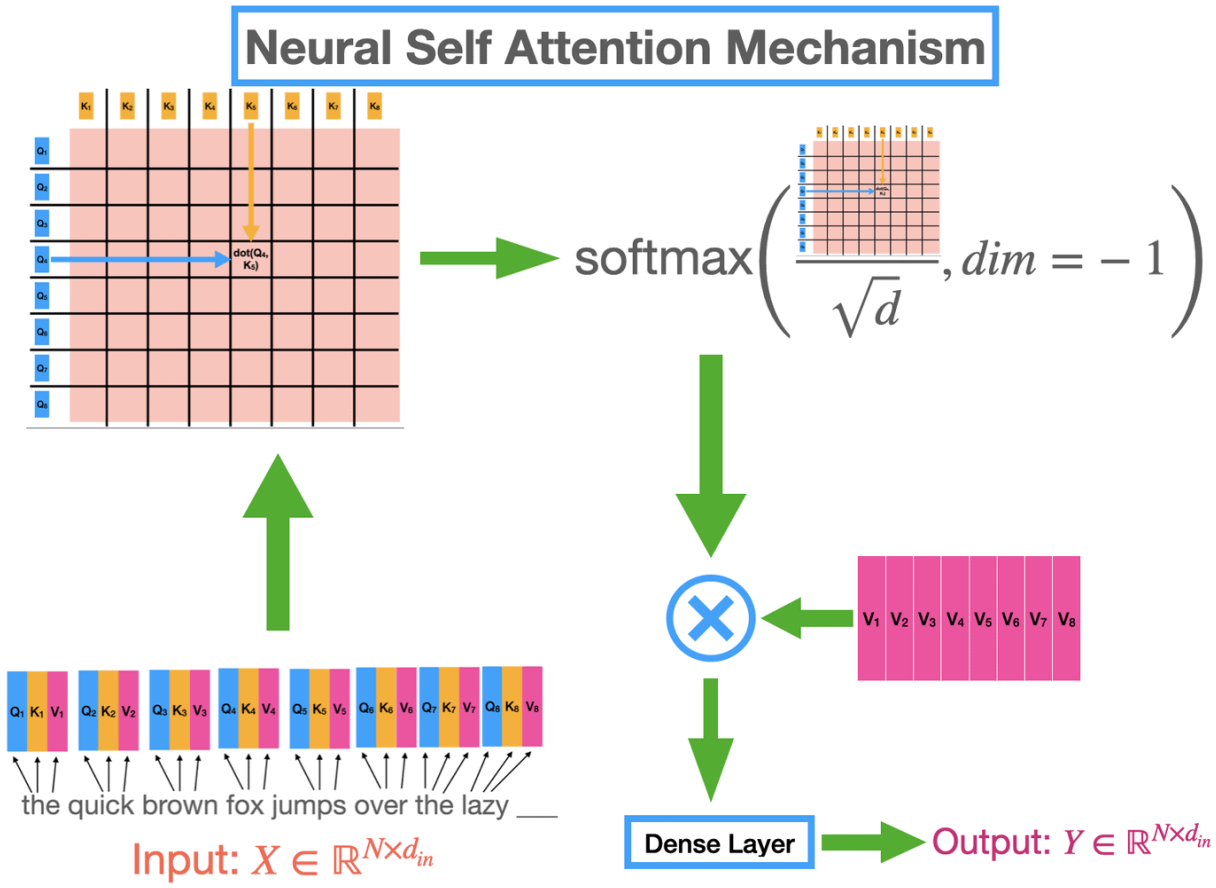


Figure 34: Mecanism Self-Attention

10.4 Arhitectura completa ViT

Transformer Encoder Block:

1. **Multi-Head Self-Attention** cu skip connection si LayerNorm
2. **MLP** (2 straturi FC) cu skip connection si LayerNorm

Formulele complete:

$$z'_l = MSA(LN(z_{l-1})) + z_{l-1} \tag{29}$$
$$z_l = MLP(LN(z'_l)) + z'_l \tag{30}$$

unde:

- MSA = Multi-Head Self-Attention
- LN = Layer Normalization
- MLP = Multi-Layer Perceptron

ViT-Base configuratie:

- Patch size: 16×16
- Hidden dimension: 768

- MLP dimension: 3072
- Heads: 12
- Layers: 12
- Parameters: 86M

10.5 Avantajele ViT fata de CNN

1. **Receptive field global:** Din primul strat, fiecare patch poate interactiona cu toate celelalte
2. **Flexibilitate:** Poate procesa imagini de diferite rezolutii fara modificari arhitecturale
3. **Interpretabilitate:** Attention maps arata ce regiuni sunt importante
4. **Scalabilitate:** Performante mai bune cu mai multe date si compute
5. **Transfer learning:** Pre-antrenare pe dataset-uri mari, fine-tuning pe task-uri specifice

10.6 Limitarile ViT

1. **Data hungry:** Necesita dataset-uri foarte mari pentru antrenare
2. **Computational cost:** Self-attention este $O(n^2)$ in numarul de patch-uri
3. **Lipsa bias-ului inductiv:** Nu "stie" despre structura spatiala a imaginilor
4. **Performante slabe pe dataset-uri mici:** CNN-urile sunt mai bune cu date limitate